

رسالة محمد

یادگیری ماشین

مدرس: محمدرضا محمدی

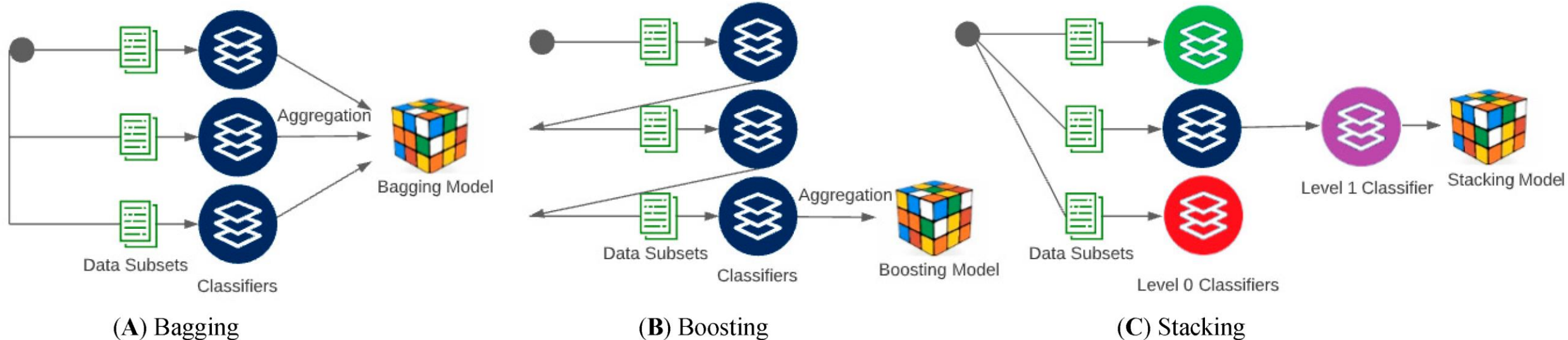
بهار ۱۴۰۲

یادگیری گروهی و جنگل تصادفی

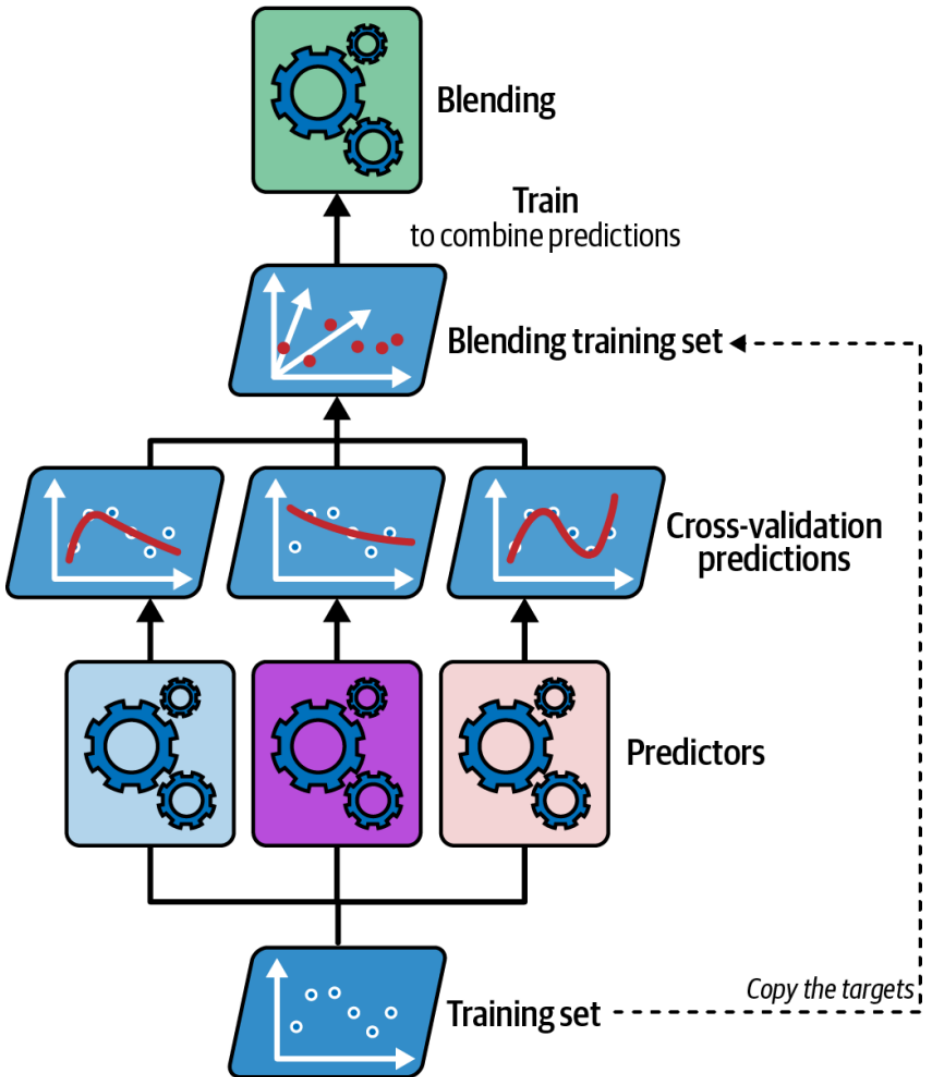
Ensemble Learning and Random Forests

Stacking

- بجای استفاده از توابع ساده مانند رای گیری برای تجميع پیش‌بینی‌ها، یک مدل آموزش داده شود - به پیش‌بینی کننده نهایی، blender (مخلوط کن) یا meta learner (فرایادگیرنده) گفته می‌شود



Stacking



```
from sklearn.ensemble import StackingClassifier

stacking_clf = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43),
    cv=5 # number of cross-validation folds
)

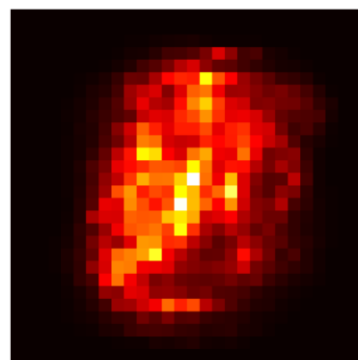
stacking_clf.fit(X_train, y_train)
```

کاهش بعد

Dimensionality Reduction

کاهش بعد

- بسیاری از مسائل یادگیری ماشین شامل هزاران یا حتی میلیون‌ها ویژگی برای هر نمونه آموزشی هستند
 - یک تصویر 224×224 رنگی به صورت خام شامل $150,528$ ویژگی است
- نه تنها باعث آموزش آهسته مدل می‌شود بلکه می‌تواند یافتن پاسخ مناسب را هم با چالش مواجه کند
 - به این مشکل، معضل بُعد (curse of dimensionality) گفته می‌شود
 - در اغلب مسائل واقعی می‌توان تعداد ویژگی‌ها را به اندازه قابل توجهی کاهش داد
 - مثلاً در MNIST بسیاری از پیکسل‌ها اهمیت کمی دارند
 - برخی پیکسل‌های مجاور هم همبستگی بالایی دارند
 - کاهش ابعاد باعث حذف مقداری از اطلاعات می‌شود و ممکن است باعث کاهش کیفیت مدل شود
 - البته اگر اطلاعات غیرضروری را کم کند می‌تواند از overfitting جلوگیری کند



Very important

Not important

معضل بُعد

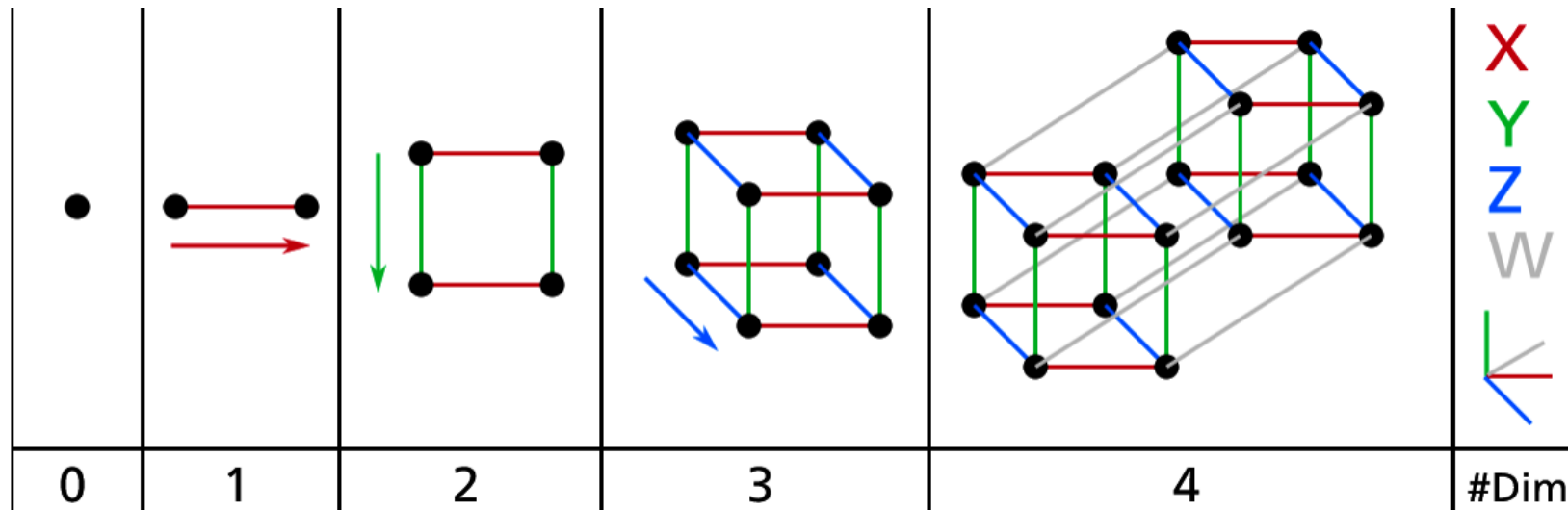
- یک نقطه تصادفی در یک مربع واحد (مربع 1×1) با چه احتمالی در فاصله کمتر از 0.001 از مرز قرار می‌گیرد؟

- $1 - 0.998 * 0.998 \simeq 0.4\%$

- در ابعاد 10,000 بعدی چطور؟

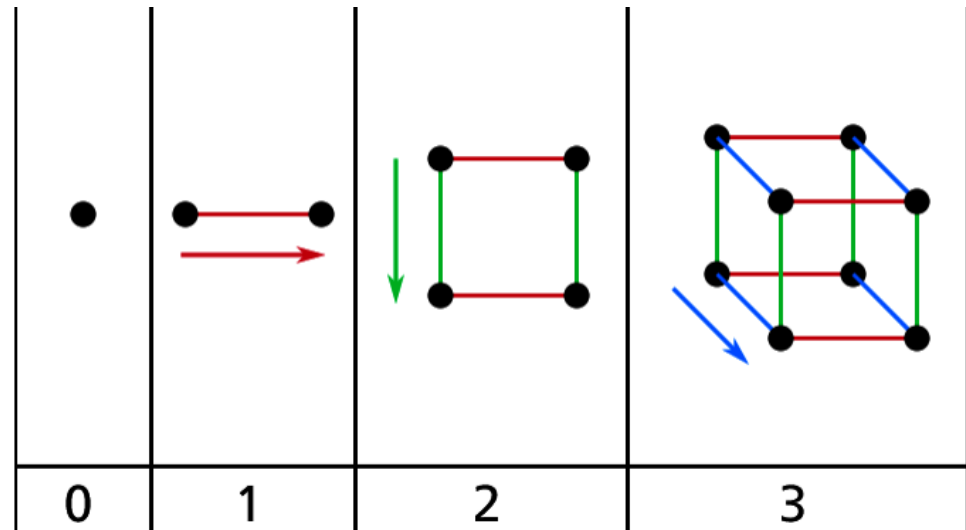
- بالاتر از 99.999999%

- در ابعاد بالا، اکثر نقاط بسیار نزدیک به مرز ابرمکعب هستند



معضل بُعد

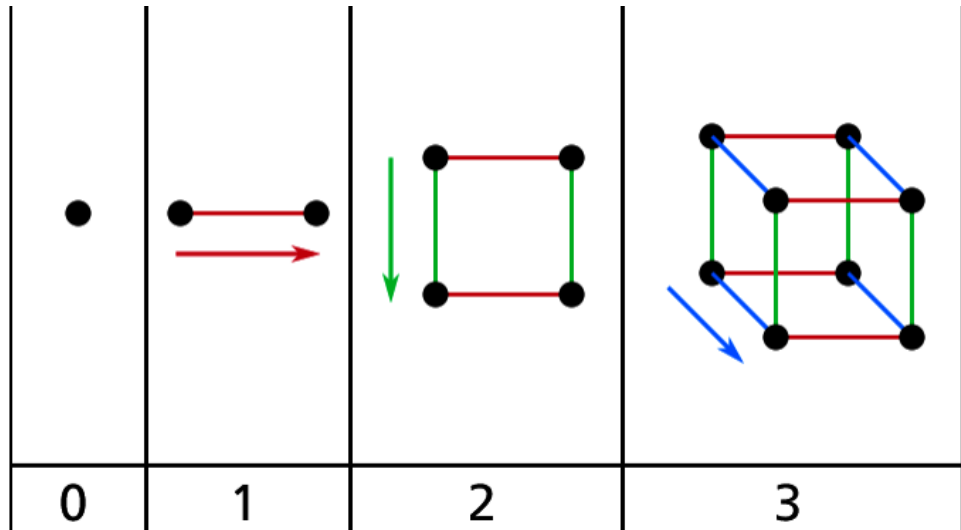
- فاصله دو نقطه تصادفی درون یک مربع واحد، به طور میانگین حدود 0.52 است
- برای مکعب واحد سه بعدی، حدود 0.66 است
- برای ابرمکعب واحد 1,000,000 بعدی، حدود 408.25 است
- به عنوان نتیجه، مجموعه داده‌های با ابعاد بالا در این خطر هستند که بسیار تنک (sparse) باشند



- اغلب نمونه‌های آموزشی احتمالاً از یکدیگر فاصله زیادی دارند
- یک نمونه جدید، احتمالاً از تمام نمونه‌های آموزشی دور خواهد بود
- پیش‌بینی‌ها در ابعاد بالا بسیار کمتر قابل اعتماد هستند
- خطر overfitting بسیار بیشتر است

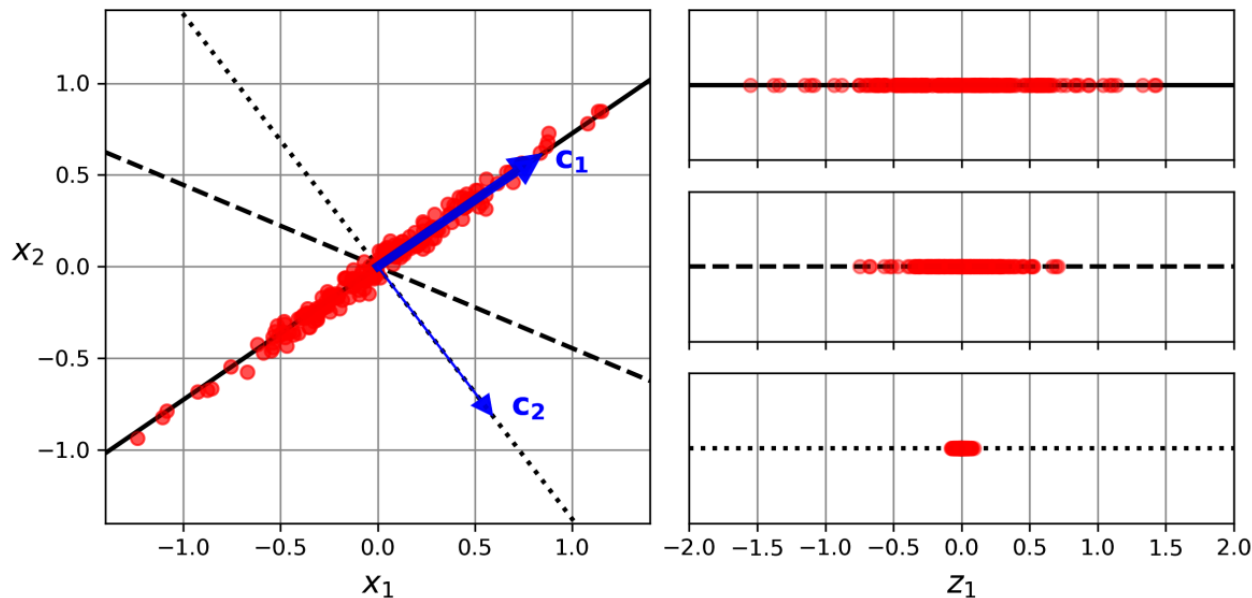
معضل بُعد

- به عنوان نتیجه، مجموعه داده‌های با ابعاد بالا در این خطر هستند که بسیار تنک (sparse) باشند
 - یک نمونه جدید، احتمالاً از تمام نمونه‌های آموزشی دور خواهد بود
 - یک راه حل تئوری این است که تعداد نمونه‌های آموزشی افزایش بیابد تا چگالی کافی حاصل شود
 - تعداد نمونه‌های آموزشی مورد نیاز برای جبران این معضل به صورت نمایی بر حسب تعداد ابعاد زیاد می‌شود
 - فقط با 100 ویژگی در بازه 0 تا 1، برای آنکه نمونه‌های آموزشی به طور میانگین در محدوده 0.1 از هم قرار داشته باشند، تعداد نمونه مورد نیاز بیش از تعداد اتم‌های قابل مشاهده در دنیا است



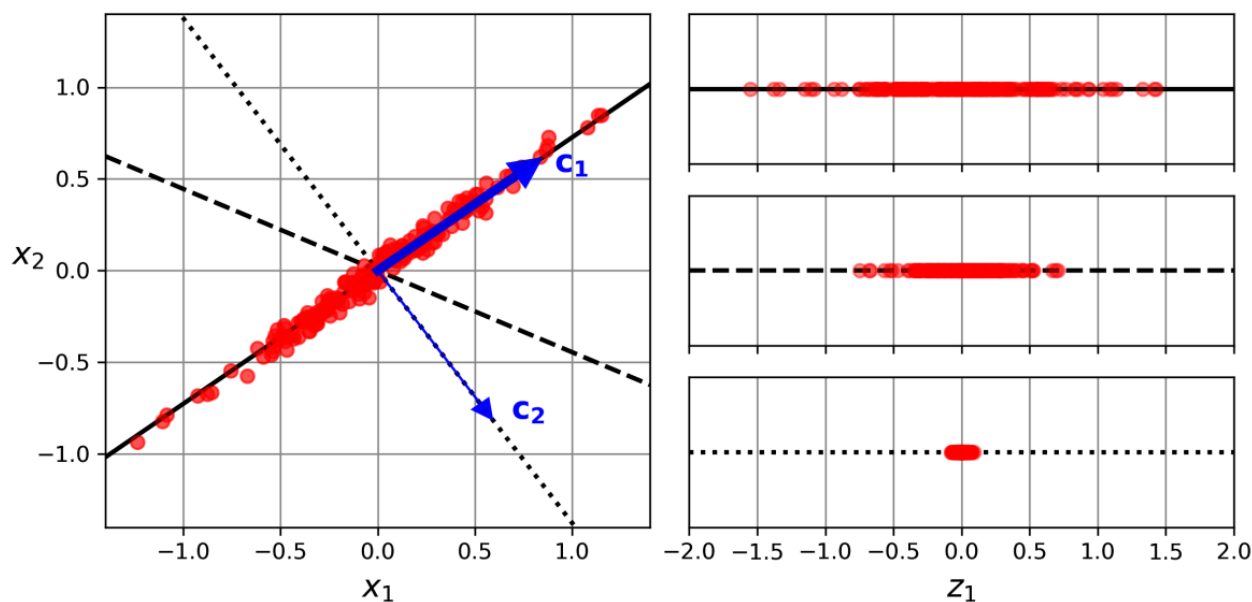
PCA

- تحلیل مولفه‌های اصلی (Principal Component Analysis) یکی از روش‌های بسیار پرکاربرد برای کاهش بعد است
- در PCA، جهت‌هایی در فضا که نمونه‌ها بیشترین تغییرات را در آن جهت‌ها دارند شناسایی می‌شوند و می‌توان هر نمونه را به این جهت‌ها افکند (projection)
- افکنش بر c_1 واریانس زیادی را حفظ کرده است
- اطلاعات کمتری را حذف کرده است



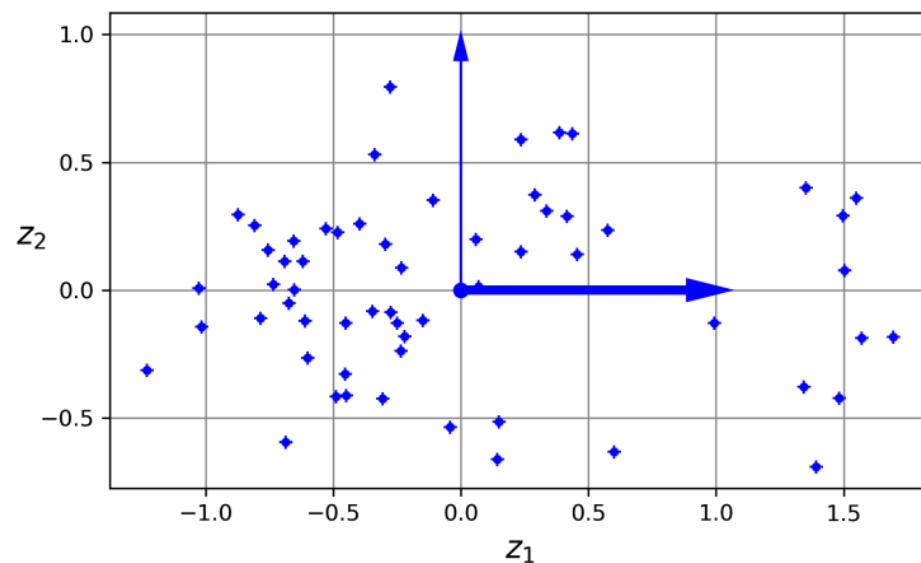
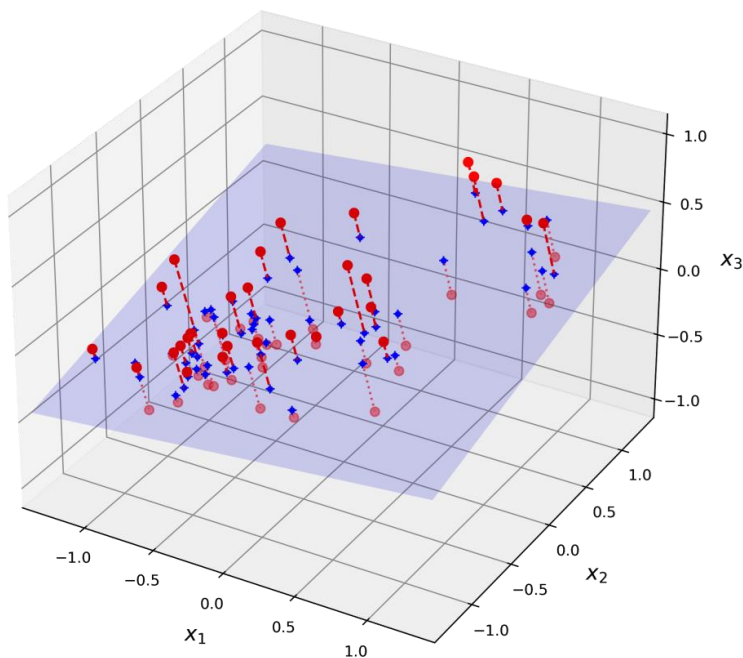
مولفه‌های اصلی

- پس از یافتن مولفه‌ای که بیشترین واریانس را حفظ می‌کند، مولفه‌های بعدی به گونه‌ای انتخاب می‌شوند که بر مولفه‌های قبلی عمود باشند و بیشترین واریانس را داشته باشند



مولفه‌های اصلی

- پس از یافتن مولفه‌ای که بیشترین واریانس را حفظ می‌کند، مولفه‌های بعدی به گونه‌ای انتخاب می‌شوند که بر مولفه‌های قبلی عمود باشند و بیشترین واریانس را داشته باشند
- چطور مولفه‌های اصلی را پیدا کنیم؟
 - باید یک مسئله بهینه‌سازی حل شود (حداکثر واریانس)



مولفه‌های اصلی

- خوشبختانه مسئله بهینه‌سازی برای یافتن مولفه‌های اصلی دارای پاسخ ساده‌ای است که با استفاده از تکنیک SVD (Singular Value Decomposition) قابل محاسبه است

- در SVD، ماتریس $\mathbf{X}$ به سه ماتریس تجزیه می‌شود: $\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$

$$\mathbf{V} = \begin{pmatrix} | & | & & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ | & | & & | \end{pmatrix}$$

- ستون‌های $\mathbf{V}$ همان مولفه‌های اصلی هستند

- برای افکندن نمونه‌ها بر ابرصفحه d بعدی، کافی است ضرب داخلی نمونه‌ها با PCها را محاسبه کنیم

```
import numpy as np
```

```
X = [...] # create a small 3D dataset
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt[0]
c2 = Vt[1]
```

```
W2 = Vt[:2].T
X2D = X_centered @ W2
```

PCA

- کلاس PCA در sklearn به طور خودکار میانگین نمونه‌ها را صفر می‌کند
- درصد واریانس مجموعه داده در راستای هر کدام از PC ها در `explained_variance_ratio_` ذخیره می‌شود
- حدود 76% از واریانس مجموعه داده در مولفه اول، حدود 15% در مولفه دوم، و حدود 9% در مولفه سوم بوده است

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)  
X2D = pca.fit_transform(X)
```

```
>>> pca.explained_variance_ratio_  
array([0.7578477 , 0.15186921])
```

انتخاب تعداد ابعاد

- می‌توان تعداد مولفه‌هایی را انتخاب کرد که بخش قابل توجهی از واریانس نمونه‌ها باقی بماند (مثلاً 95%)

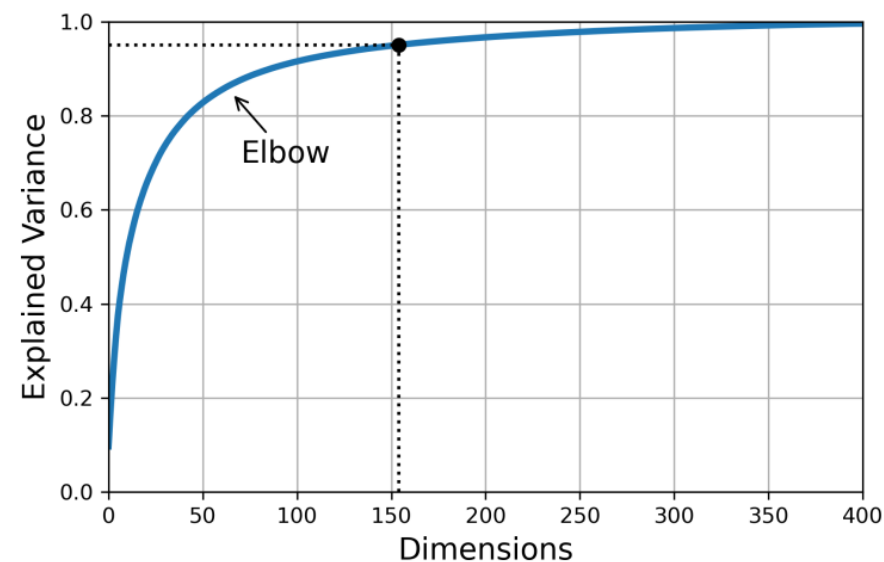
```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
X_train, y_train = mnist.data[:60_000], mnist.target[:60_000]
X_test, y_test = mnist.data[60_000:], mnist.target[60_000:]

pca = PCA()
pca.fit(X_train)
cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1 # d equals 154
```

```
pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)
```

```
>>> pca.n_components_
154
```



- در خود PCA هم می‌توان بجای تعداد، درصد را مشخص کرد

انتخاب تعداد ابعاد

- اگر از کاهش بعد به عنوان یک گام پیش‌پردازش برای یک مسئله با نظارت استفاده شود، می‌توان تعداد مولفه‌های مناسب برای آن را از طریق جستجو بدست آورد

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42),
                    RandomForestClassifier(random_state=42))
param_distrib = {
    "pca__n_components": np.arange(10, 80),
    "randomforestclassifier__n_estimators": np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3,
                                random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])

>>> print(rnd_search.best_params_)
{'randomforestclassifier__n_estimators': 465, 'pca__n_components': 23}
```

Randomized PCA

- اگر ابعاد ورودی و/یا تعداد نمونه‌ها زیاد باشد، محاسبه تمام مولفه‌های اساسی با استفاده از SVD می‌تواند بسیار زمان‌بر باشد
 - نسخه randomized می‌تواند به صورت تقریبی و با سرعت بیشتر فقط مولفه‌های مورد نظر را محاسبه کند
 - پیچیدگی محاسباتی با استفاده از SVD کامل $O(m \times n^2) + O(n^3)$ است درحالی‌که با استفاده از الگوریتم تصادفی $O(m \times d^2) + O(d^3)$ است
- ```
rnd_pca = PCA(n_components=154, svd_solver="randomized", random_state=42)
X_reduced = rnd_pca.fit_transform(X_train)
```
- به صورت پیش‌فرض، اگر  $\max(m,n) > 500$  و  $d < 0.8 \times \min(m,n)$  باشد از حالت تصادفی استفاده می‌کند و در غیر این صورت از حالت کامل استفاده می‌کند

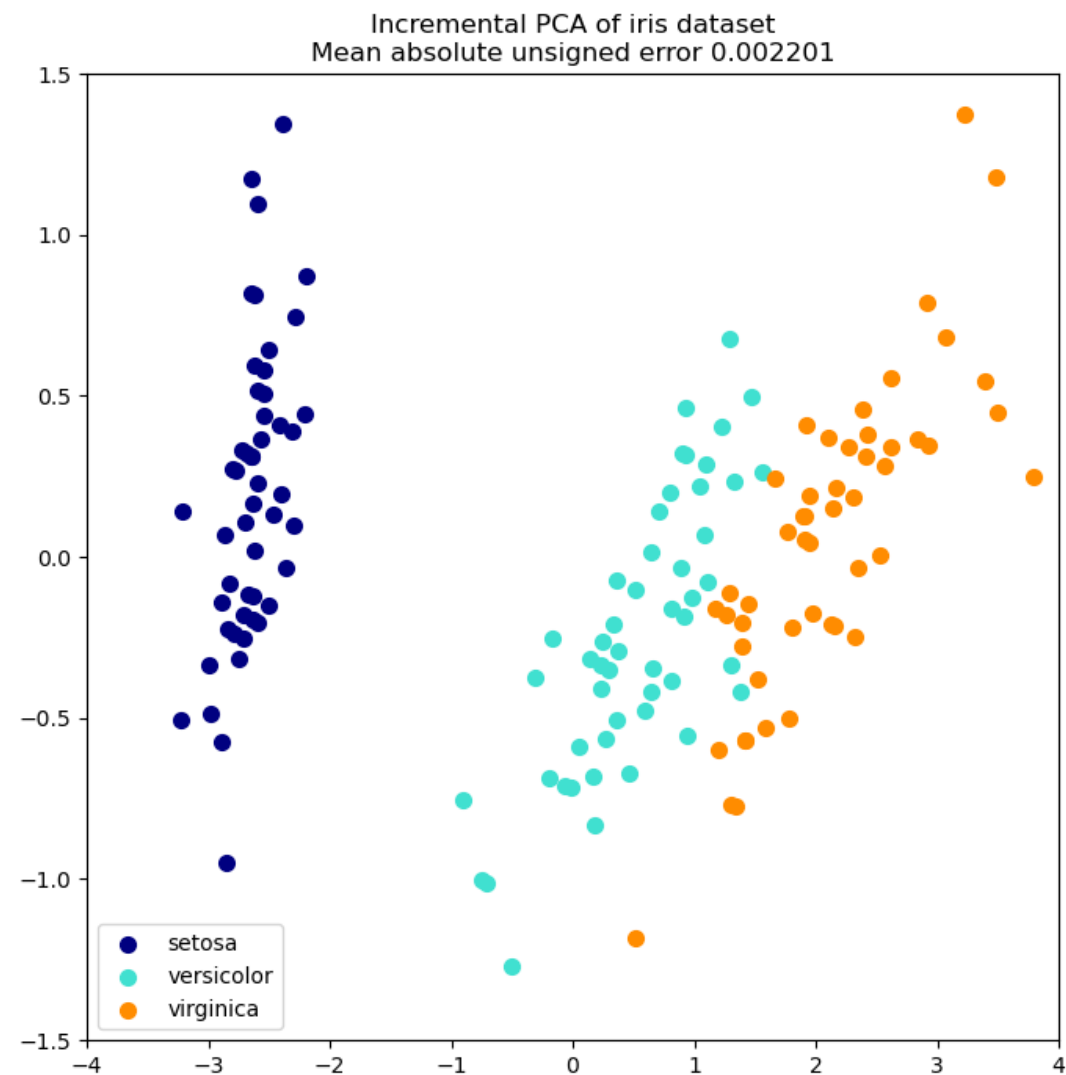
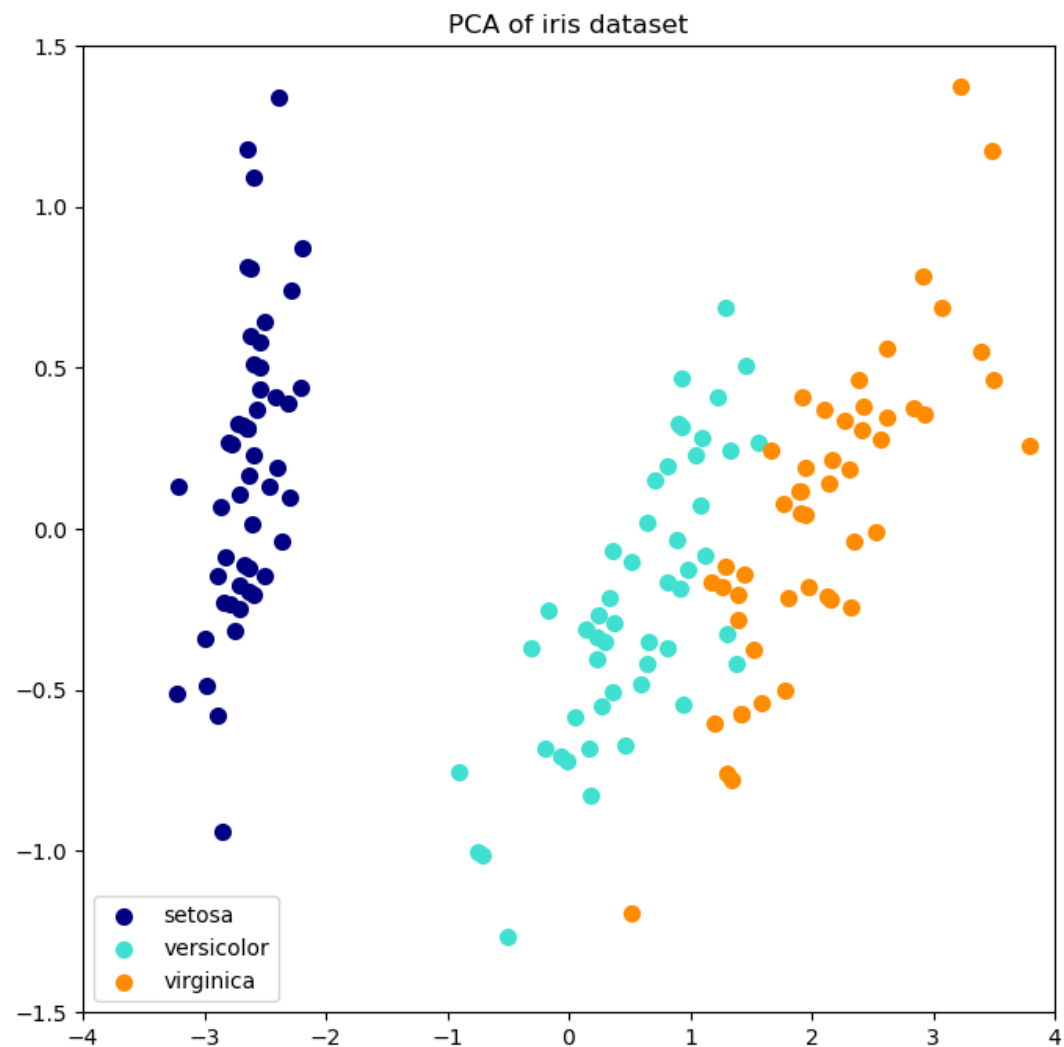
# Incremental PCA

- اگر مجموعه داده بسیار بزرگ باشد، ممکن است نتوان تمام مجموعه داده را در حافظه بارگذاری و الگوریتم را اجرا کرد
- الگوریتم IPCA توسعه داده شده است تا بتوان مجموعه داده را به بخش‌های کوچکی تقسیم کرد و مولفه‌های اصلی را به صورت تدریجی آموزش داد

```
from sklearn.decomposition import IncrementalPCA
```

```
inc_pca = IncrementalPCA(n_components=154, batch_size=1000)
```

```
X_reduced = inc_pca.fit_transform(X_train)
```



# افکنش تصادفی (Random Projection)

- برای مجموعه داده‌های بسیار بزرگ با ابعاد زیاد، حتی IPCA هم به صورت عملی قابل استفاده نیست و بسیار پرهزینه است
  - در افکنش تصادفی، از یک تابع خطی تصادفی برای کاهش بعد استفاده می‌شود
    - اگرچه جهت‌های بهینه برای حفظ واریانس نیستند، اما باز هم قابل اثبات است که تا حد خوبی می‌تواند واریانس را حفظ کند اگر تعداد ابعاد کافی باشد
    - نمونه‌های مشابه تا حد خوبی مشابه می‌مانند و تمایز نمونه‌های متفاوت تا حد خوبی حفظ می‌شود
    - توسط Johnson و Lindenstrauss اثبات شده است که با احتمال بسیار زیاد می‌توان مطمئن بود مربع فاصله نمونه‌ها کمتر از  $\epsilon\%$  تغییر می‌کند اگر ابعاد  $m$  نمونه به  $d$  کاهش داده شود
- وابسته به  $n$  نیست!

```
>>> from sklearn.random_projection import johnson_lindenstrauss_min_dim
>>> m, ε = 5_000, 0.1
>>> d = johnson_lindenstrauss_min_dim(m, eps=ε)
>>> d
7300
```

# افکنش تصادفی

- هیچ آموزشی ندارد و به راحتی و بسیار سریع قابل پیاده‌سازی است

```
n = 20_000
np.random.seed(42)
P = np.random.randn(d, n) / np.sqrt(d) # std dev = square root of variance
```

```
X = np.random.randn(m, n) # generate a fake dataset
X_reduced = X @ P.T
```

```
from sklearn.random_projection import GaussianRandomProjection
```

```
gaussian_rnd_proj = GaussianRandomProjection(eps=ε, random_state=42)
X_reduced = gaussian_rnd_proj.fit_transform(X) # same result as above
```